

Лабораторная работа 2: Разработка WPF-приложения с архитектурой Domain-UI

Введение в WPF

Что такое WPF?

WPF (Windows Presentation Foundation) — это современная платформа для создания настольных приложений под Windows. WPF использует декларативный язык разметки **XAML** для описания пользовательского интерфейса.

Основные преимущества WPF:

- **Разделение логики и представления** — интерфейс описывается в XAML, бизнес-логика — в C#
- **Data Binding** — автоматическая синхронизация данных между UI и объектами
- **Гибкость дизайна** — векторная графика, стили, анимации
- **Архитектурные паттерны** — поддержка MVVM, MVC и других паттернов

Что такое XAML?

XAML (eXtensible Application Markup Language) — это XML-подобный язык разметки для описания пользовательского интерфейса. XAML компилируется в C# код во время сборки проекта.

Пример простого окна в XAML:

```
<Window x:Class="MyApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Моё приложение" Height="450" Width="800">
    <Grid>
        <TextBlock Text="Привет, WPF!" HorizontalAlignment="Center" VerticalAlignment="Center"/>
    </Grid>
</Window>
```

Архитектура приложения

В данной лекции мы будем использовать **многопроектную архитектуру** с разделением на слои:

- **Domain** — классы предметной области (сущности, бизнес-логика)
- **UI** — пользовательский интерфейс (окна, элементы управления)

Такое разделение обеспечивает:

- Чистоту кода
- Переиспользуемость классов Domain в других проектах
- Возможность тестирования бизнес-логики независимо от UI

Создание структуры проекта

Шаг 1: Создание Solution и проектов

1. Откройте Visual Studio и создайте новый проект типа **WPF Application** (.NET 9.0)
 - Назовите проект: <ИмяПроекта>.UI (например, LibraryApp.UI, ShopApp.UI)
 - Назовите Solution: <ИмяПроекта> (например, LibraryApp, ShopApp)
2. Добавьте в Solution новый проект типа **Class Library** (.NET 9.0)

- Назовите проект: `<ИмяПроекта>.Domain` (например, `LibraryApp.Domain`)
 - Удалите автоматически созданный файл `Class1.cs`
3. Добавьте ссылку из проекта `<ИмяПроекта>.UI` на проект `<ИмяПроекта>.Domain`:
- ПКМ по проекту `<ИмяПроекта>.UI` → Add → Project Reference
 - Выберите `<ИмяПроекта>.Domain`

Начальная структура Solution:

```
<ИмяПроекта>/
├── <ИмяПроекта>.Domain/      # Классы предметной области
│   └── (пока пусто)
└── <ИмяПроекта>.UI/         # Пользовательский интерфейс
    ├── App.xaml
    ├── App.xaml.cs
    ├── MainWindow.xaml
    ├── MainWindow.xaml.cs
    └── AssemblyInfo.cs
```

Итоговая структура Solution (после выполнения всех шагов):

```
<ИмяПроекта>/
├── <ИмяПроекта>.Domain/      # Классы предметной области
│   ├── <Сущность1>.cs
│   ├── <Сущность2>.cs
│   ├── <Сущность3>.cs
│   └── <Enum>.cs
└── <ИмяПроекта>.UI/         # Пользовательский интерфейс
    ├── App.xaml
    ├── MainWindow.xaml
    ├── <Окно>EditWindow.xaml
    └── Converters/
        ├── <Конвертер1>.cs
        ├── <Конвертер2>.cs
        └── ...
```

Создание Domain-слоя

Domain-слой содержит классы, которые описывают **предметную область** приложения.

Пример предметной области: Система записи клиентов в салон красоты. Мы создадим классы для клиентов, мастеров и резерваций.

Ваша задача: Определите 2-4 класса-сущности на основе вашего задания и создайте их аналогично примерам ниже.

Пример: Класс Client

Класс `Client` представляет клиента салона красоты.

Файл: `<ИмяПроекта>.Domain/Client.cs`

```
namespace <ИмяПроекта>.Domain;

public class Client
{
    public required string Name { get; set; }
    public required string Phone { get; set; }
}
```

Разбор кода:

- `namespace <ИмяПроекта>.Domain;` — пространство имён проекта (замените на имя вашего проекта)
- `required` — модификатор C# 11, означающий, что свойство обязательно должно быть инициализировано при создании объекта

Пример: Класс Master

Класс `Master` представляет мастера, который предоставляет услуги.

Файл: `<ИмяПроекта>.Domain/Master.cs`

```
namespace <ИмяПроекта>.Domain;

public class Master
{
    public required Guid Id { get; set; }
    public required string Name { get; set; }
}
```

Разбор кода:

- `Guid` — глобально уникальный идентификатор (GUID), используется для уникальной идентификации объектов

Пример: Enum ReservationStatus

Перечисление `ReservationStatus` описывает возможные статусы резервации.

Файл: `<ИмяПроекта>.Domain/ReservationStatus.cs`

```
namespace <ИмяПроекта>.Domain;

public enum ReservationStatus
{
    Created,          // Создана
    Confirmed,        // Подтверждена
    InProgress,       // В процессе
    Cancelled,         // Отменена
    Done,              // Завершена
}
```

Пример: Класс Reservation

Класс `Reservation` представляет запись клиента к мастеру.

Файл: `<ИмяПроекта>.Domain/Reservation.cs`

```
namespace <ИмяПроекта>.Domain;

public class Reservation
{
    public required Guid Id { get; set; }
    public required Client Client { get; set; }
    public required Master? Master { get; set; }
    public required string ServiceDescription { get; set; }
    public required DateOnly Date { get; set; }
    public required TimeOnly StartTime { get; set; }
    public required TimeSpan Duration { get; set; }
    public required ReservationStatus Status { get; set; }

    public static Reservation Create()
    {
        var now = DateTime.Now;
        return new Reservation
        {
            Id = Guid.NewGuid(),

```

```

        Client = new Client { Name = "", Phone = "" },
        Master = null,
        ServiceDescription = "",
        Date = DateOnly.FromDateTime(now),
        StartTime = TimeOnly.FromDateTime(now),
        Duration = TimeSpan.FromMinutes(30),
        Status = ReservationStatus.Created
    };
}

public Reservation Clone()
{
    return new Reservation
    {
        Id = Id,
        Client = new Client
        {
            Name = Client.Name,
            Phone = Client.Phone
        },
        Master = Master,
        ServiceDescription = ServiceDescription,
        Date = Date,
        StartTime = StartTime,
        Duration = Duration,
        Status = Status
    };
}
}

```

Разбор кода:

- Master? — nullable-тип, означает, что мастер может быть не назначен
- DateOnly — тип C# 10 для хранения только даты (без времени)
- TimeOnly — тип C# 10 для хранения только времени (без даты)
- TimeSpan — представляет интервал времени (длительность)
- Create() — статический метод-фабрика для создания новой резервации с начальными значениями
- Clone() — метод для создания копии объекта (нужен для редактирования без изменения оригинала)

Создание UI-слоя

Основы XAML и Window

Каждое окно WPF состоит из двух файлов:

- **XAML-файл** (например, MainWindow.xaml) — описание интерфейса
- **Code-behind файл** (например, MainWindow.xaml.cs) — логика обработки событий

Создание формы редактирования

Создадим окно для создания и редактирования сущности (в примере — резервации).

Примечание: Название окна, пространства имён и классы замените на свои согласно вашей предметной области.

Файл: <ИмяПроекта>.UI/<Сущность>EditWindow.xaml (например, ReservationEditWindow.xaml)

```

<Window x:Class="<ИмяПроекта>.UI.<Сущность>EditWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:local="clr-namespace:<ИмяПроекта>.UI"

```

```

xmlns:converters="clr-namespace:<ИмяПроекта>.UI.Converters"
xmlns:domain="clr-namespace:<ИмяПроекта>.Domain;assembly=<ИмяПроекта>.Domain"
mc:Ignorable="d"
Title="Редактирование" Height="450" Width="800">
<Window.DataContext>
    <domain:<ВашКласс>/>
</Window.DataContext>
<Window.Resources>
    <!-- Здесь объявляются конвертеры, которые мы создадим позже -->
    <converters:<Ваш>Converter x:Key="<Ключ>Converter"/>
</Window.Resources>
<Grid Margin="20">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <!-- Добавьте нужное количество строк для ваших полей -->
        <RowDefinition Height="*/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="150"/>
        <ColumnDefinition Width="*/>
    </Grid.ColumnDefinitions>

    <!-- Пример: Текстовое поле -->
    <TextBlock Grid.Row="0" Grid.Column="0" Text="Название поля:" VerticalAlignment="Center"
Margin="0,0,0,10"/>
    <TextBox Grid.Row="0" Grid.Column="1" Text="{Binding <СвойствоОбъекта>}" Margin="0,0,0,10"/>

    <!-- Пример: ComboBox для выбора из списка -->
    <TextBlock Grid.Row="1" Grid.Column="0" Text="Выберите:" VerticalAlignment="Center"
Margin="0,0,0,10"/>
    <ComboBox Grid.Row="1" Grid.Column="1"
        ItemsSource="{x:Static local:<ВашеОкно>.Available<Элементы>}"
        SelectedItem="{Binding <СвойствоСсылка>}"
        DisplayMemberPath="<СвойствоОтображения>"
        Margin="0,0,0,10"/>

    <!-- Пример: DatePicker с конвертером -->
    <!-- <TextBlock Grid.Row="2" Grid.Column="0" Text="Дата:" VerticalAlignment="Center"
Margin="0,0,0,10"/>
    <DatePicker Grid.Row="2" Grid.Column="1" SelectedDate="{Binding <Дата>, Converter={StaticResource
DateOnlyConverter}}}" Margin="0,0,0,10"/> -->

    <!-- Кнопки -->
    <StackPanel Grid.Row="3" Grid.Column="0" Grid.ColumnSpan="2" Orientation="Horizontal"
HorizontalAlignment="Right" Margin="0,10,0,0">
        <Button Content="Сохранить" Width="100" Margin="0,0,10,0" Click="SaveButton_Click"/>
        <Button Content="Отмена" Width="100" Click="CancelButton_Click"/>
    </StackPanel>
</Grid>
</Window>

```

Полный пример для резервации салона красоты:

Ниже приведён полный код XAML для окна редактирования резервации, который вы можете адаптировать для своей предметной области:

```

<Window x:Class="BeautyApp.UI.ReservationEditWindow"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="clr-namespace:BeautyApp.UI"
xmlns:converters="clr-namespace:BeautyApp.UI.Converters"
xmlns:domain="clr-namespace:BeautyApp.Domain;assembly=BeautyApp.Domain"
Title="Редактирование резервации" Height="450" Width="800">

```

```

<Window.Resources>
    <converters:ReservationStatusConverter x:Key="StatusConverter"/>
    <converters:DateOnlyConverter x:Key="DateOnlyConverter"/>
    <converters:TimeOnlyConverter x:Key="TimeOnlyConverter"/>
</Window.Resources>
<Grid Margin="20">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="150"/>
        <ColumnDefinition Width="*/>
    </Grid.ColumnDefinitions>

    <TextBlock Grid.Row="0" Grid.Column="0" Text="Имя клиента:" VerticalAlignment="Center"
Margin="0,0,0,10"/>
    <TextBox Grid.Row="0" Grid.Column="1" Text="{Binding Client.Name}" Margin="0,0,0,10"/>

    <TextBlock Grid.Row="1" Grid.Column="0" Text="Телефон:" VerticalAlignment="Center"
Margin="0,0,0,10"/>
    <TextBox Grid.Row="1" Grid.Column="1" Text="{Binding Client.Phone}" Margin="0,0,0,10"/>

    <TextBlock Grid.Row="2" Grid.Column="0" Text="Мастер:" VerticalAlignment="Center"
Margin="0,0,0,10"/>
    <ComboBox Grid.Row="2" Grid.Column="1"
        ItemsSource="{x:Static local:ReservationEditWindow.AvailableMasters}"
        SelectedItem="{Binding Master}"
        DisplayMemberPath="Name"
        Margin="0,0,0,10"/>

    <TextBlock Grid.Row="3" Grid.Column="0" Text="Дата:" VerticalAlignment="Center"
Margin="0,0,0,10"/>
    <DatePicker Grid.Row="3" Grid.Column="1" SelectedDate="{Binding Date, Converter={StaticResource
DateOnlyConverter}}" Margin="0,0,0,10"/>

    <TextBlock Grid.Row="4" Grid.Column="0" Text="Статус:" VerticalAlignment="Center"
Margin="0,0,0,10"/>
    <ComboBox Grid.Row="4" Grid.Column="1"
        ItemsSource="{x:Static local:ReservationEditWindow.AvailableStatuses}"
        SelectedItem="{Binding Status}"
        Margin="0,0,0,10">
        <ComboBox.ItemTemplate>
            <DataTemplate>
                <TextBlock Text="{Binding Converter={StaticResource StatusConverter}}"/>
            </DataTemplate>
        </ComboBox.ItemTemplate>
    </ComboBox>

    <StackPanel Grid.Row="6" Grid.Column="0" Grid.ColumnSpan="2" Orientation="Horizontal"
HorizontalAlignment="Right" Margin="0,10,0,0">
        <Button Content="Сохранить" Width="100" Margin="0,0,10,0" Click="SaveButton_Click"/>
        <Button Content="Отмена" Width="100" Click="CancelButton_Click"/>
    </StackPanel>
</Grid>
</Window>

```

Разбор ключевых элементов XAML:

1. Data Binding (Привязка данных)

Data Binding — это основной механизм связывания UI с данными. Синтаксис: `{Binding <Путь>}`

Примеры привязки к свойствам:

```
<!-- Простое свойство -->
<TextBox Text="{Binding Name}"/>

<!-- Вложенное свойство -->
<TextBox Text="{Binding Client.Name}"/>

<!-- Числовое свойство -->
<TextBox Text="{Binding Age}"/>
```

Важно: Привязка работает автоматически — изменения в `TextBox` сразу попадают в объект, и наоборот.

2. ComboBox с привязкой к списку

`ComboBox` используется для выбора элемента из списка:

```
<ComboBox ItemsSource="{x:Static local:ВашеОкно.Available<Список>}"
  SelectedItem="{Binding <СвойствоОбъекта>}"
  DisplayMemberPath="<СвойствоОтображения>"/>
```

Разбор:

- `ItemsSource` — откуда берётся список элементов (привязка к статическому свойству)
- `SelectedItem` — какой элемент выбран (привязывается к свойству вашего объекта)
- `DisplayMemberPath` — какое свойство элемента показывать в списке (например, "Name")

Пример:

```
<!-- Выбор мастера из списка -->
<ComboBox ItemsSource="{x:Static local:ReservationEditWindow.AvailableMasters}"
  SelectedItem="{Binding Master}"
  DisplayMemberPath="Name"/>
```

3. Конвертеры в привязках

Конвертеры преобразуют значения между объектом и UI. Используются, когда типы не совпадают.

Синтаксис:

```
<Element Property="{Binding <Свойство>, Converter={StaticResource <КлючКонвертера>}}"/>
```

Примеры:

```
<!-- DateOnly → DateTime (для DatePicker) -->
<DatePicker SelectedDate="{Binding Date, Converter={StaticResource DateOnlyConverter}}"/>

<!-- TimeOnly → string (для TextBox) -->
<TextBox Text="{Binding StartTime, Converter={StaticResource TimeOnlyConverter}}"/>

<!-- Enum → string на русском (для ComboBox) -->
<ComboBox ItemsSource="{x:Static local:Window.AvailableStatuses}"
  SelectedItem="{Binding Status}">
  <ComboBox.ItemTemplate>
    <DataTemplate>
      <TextBlock Text="{Binding Converter={StaticResource StatusConverter}}"/>
    </DataTemplate>
  </ComboBox.ItemTemplate>
</ComboBox>
```

Важно: Конвертеры должны быть объявлены в `Window.Resources` (см. следующий раздел).

Data Binding и DataContext

Что такое Data Binding?

Data Binding — это механизм автоматической синхронизации данных между элементами UI и объектами C#. Когда изменяется значение в `TextBox`, автоматически изменяется соответствующее свойство объекта, и наоборот.

DataContext

DataContext — это контекст данных, к которому привязываются элементы управления. `DataContext` может быть установлен:

- В XAML (как в примере выше)
- В коде C#

Установка DataContext в коде

Файл: `<ИмяПроекта>.UI/<Сущность>EditWindow.xaml.cs`

Ниже приведён упрощённый шаблон code-behind файла. Адаптируйте его под свою предметную область:

```
namespace <ИмяПроекта>.UI;

using System.Collections.ObjectModel;
using System.Windows;
using <ИмяПроекта>.Domain;

public partial class <Сущность>EditWindow : Window
{
    private <ВашКласс> _entity;

    // Статическая коллекция для ComboBox (если нужно)
    public static ObservableCollection<<Связанный>Класс> Available<Элементы> { get; set; } = new()
    {
        // Здесь добавьте элементы для выпадающего списка
    };

    // Статический массив для Enum (если используется)
    public static <ВашEnum>[] Available<Статусы> { get; } = Enum.GetValues<<ВашEnum>>();

    private <Сущность>EditWindow(<ВашКласс>? entity)
    {
        _entity = entity is null
            ? <ВашКласс>.Create() // Создание нового объекта
            : entity.Clone();      // Клонирование существующего
        InitializeComponent();
        DataContext = _entity;    // Установка DataContext
    }

    public static <ВашКласс>? Create()
    {
        var window = new <Сущность>EditWindow(null);
        window.Title = "Создание";

        return window.ShowDialog() == true ? window._entity : null;
    }

    public static <ВашКласс>? Edit(<ВашКласс> entity)
    {
        var window = new <Сущность>EditWindow(entity);
        window.Title = "Редактирование";
    }
}
```



```

        return window.ShowDialog() == true ? window._entity : null;
    }

    private void SaveButton_Click(object sender, RoutedEventArgs e)
    {
        // Здесь добавьте валидацию, если нужно
        DialogResult = true;
        Close();
    }

    private void CancelButton_Click(object sender, RoutedEventArgs e)
    {
        DialogResult = false;
        Close();
    }
}

```

Полный пример code-behind для резервации:

```

namespace BeautyApp.UI;

using System.Collections.ObjectModel;
using System.Windows;
using BeautyApp.Domain;
using BeautyApp.UI.Converters;

public partial class ReservationEditWindow : Window
{
    private Reservation _reservation;

    public static ObservableCollection<Master> AvailableMasters { get; set; } = new()
    {
        new Master { Id = Guid.NewGuid(), Name = "Анна Иванова" },
        new Master { Id = Guid.NewGuid(), Name = "Мария Петрова" },
        new Master { Id = Guid.NewGuid(), Name = "Елена Сидорова" }
    };

    public static ReservationStatus[] AvailableStatuses { get; } = Enum.GetValues<ReservationStatus>();

    private ReservationEditWindow(Reservation? reservation)
    {
        _reservation = reservation is null
            ? Reservation.Create()
            : reservation.Clone();
        InitializeComponent();
        DataContext = _reservation;
    }

    public static Reservation? Create()
    {
        var window = new ReservationEditWindow(null);
        window.Title = "Создание резервации";
        return window.ShowDialog() == true ? window._reservation : null;
    }

    public static Reservation? Edit(Reservation reservation)
    {
        var window = new ReservationEditWindow(reservation);
        window.Title = "Редактирование резервации";
        return window.ShowDialog() == true ? window._reservation : null;
    }

    private void SaveButton_Click(object sender, RoutedEventArgs e)
    {

```

```

        if (!ValidateForm(out var validationError))
        {
            MessageBox.Show(validationError, "Ошибка валидации", MessageBoxButton.OK,
                MessageBoxImage.Warning);
            return;
        }

        DialogResult = true;
        Close();
    }

    private bool ValidateForm(out string validationError)
    {
        validationError = string.Empty;

        if (string.IsNullOrEmpty(_reservation.Client.Name))
        {
            validationError = "Укажите имя клиента";
            return false;
        }

        if (_reservation.Master is null)
        {
            validationError = "Выберите мастера";
            return false;
        }

        return true;
    }

    private void CancelButton_Click(object sender, RoutedEventArgs e)
    {
        DialogResult = false;
        Close();
    }
}

```

Разбор кода:

1. ObservableCollection:

- ObservableCollection — коллекция, которая уведомляет UI об изменениях (добавлении/удалении элементов)
- Используется для привязки к ComboBox.ItemsSource

2. Статические методы Create и Edit:

- Паттерн **Named Constructor** — статические методы для создания окна в разных режимах
- Возвращают null, если пользователь отменил операцию

3. ShowDialog():

```
return window.ShowDialog() == true ? window._reservation : null;
```

- ShowDialog() — открывает окно модально (блокирует основное окно)
- Возвращает true, если была нажата кнопка "Сохранить", иначе false или null

4. DialogResult:

```
DialogResult = true;
```

- Устанавливает результат диалога, который возвращается из ShowDialog()

Конвертеры значений

Зачем нужны конвертеры?

Конвертеры нужны, когда тип данных в объекте не совпадает с типом, который требует элемент UI.

Примеры:

- `DatePicker` работает с `DateTime`, а у вас в классе `DateOnly` → нужен конвертер
- `TextBox` работает со `string`, а у вас `TimeOnly` → нужен конвертер
- `ComboBox` показывает enum на английском, а нужно на русском → нужен конвертер

Как работают конвертеры?

Конвертер — это класс, реализующий интерфейс **IValueConverter** с двумя методами:

- `Convert` — преобразование из объекта в значение для UI (`C#` → `XAML`)
- `ConvertBack` — преобразование из UI обратно в объект (`XAML` → `C#`)

Как подключить конвертер?

Шаг 1: Создайте класс конвертера в папке `Converters`

Шаг 2: Подключите пространство имён в `XAML`:

```
<Window ...
    xmlns:converters="clr-namespace:<ИмяПроекта>.UI.Converters">
```

Шаг 3: Объявите конвертер в ресурсах окна:

```
<Window.Resources>
    <converters:<Ваш>Converter x:Key="<Ключ>Converter"/>
</Window.Resources>
```

Шаг 4: Используйте конвертер в привязке:

```
<DatePicker SelectedDate="{Binding Date, Converter={StaticResource <Ключ>Converter}}"/>
```

Полный пример подключения

```
<Window x:Class="MyApp.UI.MyWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:converters="clr-namespace:MyApp.UI.Converters">
    <Window.Resources>
        <!-- Объявление конвертеров -->
        <converters:DateOnlyConverter x:Key="DateOnlyConverter"/>
        <converters:TimeOnlyConverter x:Key="TimeOnlyConverter"/>
        <converters:StatusConverter x:Key="StatusConverter"/>
    </Window.Resources>
    <Grid>
        <!-- Использование конвертеров -->
        <DatePicker SelectedDate="{Binding Date, Converter={StaticResource DateOnlyConverter}}"/>
        <TextBox Text="{Binding Time, Converter={StaticResource TimeOnlyConverter}}"/>
    </Grid>
</Window>
```

Интерфейс IValueConverter

Каждый конвертер должен реализовать интерфейс **IValueConverter** с двумя методами:

- `Convert` — преобразование из объекта в значение для UI
- `ConvertBack` — преобразование из UI обратно в объект

Пример: DateOnlyConverter

Конвертер для преобразования `DateOnly` в `DateTime` (для `DatePicker`).

Файл: <ИмяПроекта>.UI/Converters/DateOnlyConverter.cs

```
namespace <ИмяПроекта>.UI.Converters;

using System.Globalization;
using System.Windows;
using System.Windows.Data;

public class DateOnlyConverter : IValueConverter
{
    public object? Convert(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        if (value is DateOnly dateOnly)
        {
            return dateOnly.ToDateTime(TimeOnly.MinValue);
        }

        return DependencyProperty.UnsetValue;
    }

    public object? ConvertBack(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        if (value is DateTime dateTime)
        {
            return DateOnly.FromDateTime(dateTime);
        }

        return DependencyProperty.UnsetValue;
    }
}
```

Разбор:

- Convert — из DateOnly в DateTime (для отображения в DatePicker)
- ConvertBack — из DateTime обратно в DateOnly
- DependencyProperty.UnsetValue — возвращается, если преобразование невозможно

Пример: TimeOnlyConverter

Конвертер для преобразования TimeOnly в строку формата "ЧЧ:ММ".

Файл: <ИмяПроекта>.UI/Converters/TimeOnlyConverter.cs

```
namespace <ИмяПроекта>.UI.Converters;

using System.Globalization;
using System.Windows;
using System.Windows.Data;

public class TimeOnlyConverter : IValueConverter
{
    public object? Convert(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        if (value is TimeOnly timeOnly)
        {
            return timeOnly.ToString("HH:mm", CultureInfo.CurrentCulture);
        }

        return DependencyProperty.UnsetValue;
    }

    public object? ConvertBack(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        if (value is string text && TryParse(text, out var timeOnly))
        {

```

```

        return timeOnly;
    }

    return DependencyProperty.UnsetValue;
}

public static bool TryParse(string text, out TimeOnly timeOnly)
{
    return TimeOnly.TryParseExact(text, "HH:mm", CultureInfo.InvariantCulture, DateTimeStyles.None,
    out timeOnly);
}
}

```

Разбор:

- ToString("HH:mm") — формат времени с ведущими нулями (например, "09:30")
- TryParse — статический метод для использования в валидации

Пример: TimeSpanConverter

Конвертер для преобразования TimeSpan в строку "ЧЧ:ММ" или минуты.

Файл: <ИмяПроекта>.UI/Converters/TimeSpanConverter.cs

```

namespace <ИмяПроекта>.UI.Converters;

using System.Globalization;
using System.Windows;
using System.Windows.Data;

public class TimeSpanConverter : IValueConverter
{
    public object? Convert(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        if (value is TimeSpan timeSpan)
        {
            return $"{(int)timeSpan.TotalHours:D2}:{timeSpan.Minutes:D2}";
        }

        return DependencyProperty.UnsetValue;
    }

    public object? ConvertBack(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        if (value is string text && TryParse(text, out var timeSpan))
        {
            return timeSpan;
        }

        return DependencyProperty.UnsetValue;
    }

    public static bool TryParse(string text, out TimeSpan timeSpan)
    {
        // Формат "ЧЧ:ММ"
        if (TimeSpan.TryParseExact(text, @"hh\:mm", CultureInfo.InvariantCulture, out timeSpan))
        {
            return true;
        }

        // Формат "Ч:ММ"
        if (TimeSpan.TryParseExact(text, @"h\:mm", CultureInfo.InvariantCulture, out timeSpan))
        {
            return true;
        }
    }
}

```

```

        // Только минуты (например, "30")
        if (int.TryParse(text, out var minutes))
        {
            timeSpan = TimeSpan.FromMinutes(minutes);
            return true;
        }

        timeSpan = TimeSpan.Zero;
        return false;
    }
}

```

Разбор:

- TotalHours:D2 — общее количество часов с ведущими нулями
- Minutes:D2 — минуты с ведущими нулями
- TryParse поддерживает три формата ввода:
 - "ЧЧ:ММ" (например, "02:30")
 - "Ч:ММ" (например, "2:30")
 - Только минуты (например, "30")

Пример: Конвертер для Enum

Конвертер для преобразования enum в читаемую строку на русском языке (например, `ReservationStatus`).

Файл: <ИмяПроекта>.UI/Converters/<Ваш>StatusConverter.cs

```

namespace <ИмяПроекта>.UI.Converters;

using System.Globalization;
using System.Windows;
using System.Windows.Data;
using <ИмяПроекта>.Domain;

public class <Ваш>StatusConverter : IValueConverter
{
    public object? Convert(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        if (value is <ВашEnum> status)
        {
            return status switch
            {
                <ВашEnum>.Значение1 => "Русское название 1",
                <ВашEnum>.Значение2 => "Русское название 2",
                <ВашEnum>.Значение3 => "Русское название 3",
                _ => status.ToString()
            };
        }

        return DependencyProperty.UnsetValue;
    }

    public object? ConvertBack(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        return DependencyProperty.UnsetValue;
    }
}

```

Конкретный пример для ReservationStatus:

```

namespace BeautyApp.UI.Converters;

using System.Globalization;

```

```

using System.Windows;
using System.Windows.Data;
using BeautyApp.Domain;

public class ReservationStatusConverter : IValueConverter
{
    public object? Convert(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        if (value is ReservationStatus status)
        {
            return status switch
            {
                ReservationStatus.Created => "Создана",
                ReservationStatus.Confirmed => "Подтверждена",
                ReservationStatus.InProgress => "В процессе",
                ReservationStatus.Cancelled => "Отменена",
                ReservationStatus.Done => "Завершена",
                _ => status.ToString()
            };
        }

        return DependencyProperty.UnsetValue;
    }

    public object? ConvertBack(object? value, Type targetType, object? parameter, CultureInfo culture)
    {
        return DependencyProperty.UnsetValue;
    }
}

```

Разбор:

- `switch expression` — компактный синтаксис для преобразования значений
- `ConvertBack` возвращает `UnsetValue`, т.к. обратное преобразование не требуется (используется `SelectedItem` напрямую)

Валидация данных

Валидация — это проверка корректности введённых данных перед сохранением. В данном примере валидация реализована явно в методе `ValidateForm`.

Примечание: Адаптируйте проверки под свою предметную область.

Пример валидации:

```

private bool ValidateForm(out string validationError)
{
    validationError = string.Empty;

    // Проверка обязательных текстовых полей
    if (string.IsNullOrEmpty(_entity.<СвойствоТекст>))
    {
        validationError = "Укажите <название поля>";
        return false;
    }

    // Проверка обязательных ссылок
    if (_entity.<СвойствоСсылка> is null)
    {
        validationError = "Выберите <название поля>";
        return false;
    }
}

```

```
// Проверка формата (если используются конвертеры с TryParse)
if (!<Конвертер>.TryParse(<ТекстБокс>.Text, out _))
{
    validationError = "Неверный формат <поля>. Используйте формат <пример>.";
    return false;
}

return true;
}
```

Полный пример валидации для резервации:

```
private bool ValidateForm(out string validationError)
{
    validationError = string.Empty;

    if (string.IsNullOrEmpty(_reservation.Client.Name))
    {
        validationError = "Укажите имя клиента";
        return false;
    }

    if (string.IsNullOrEmpty(_reservation.Client.Phone))
    {
        validationError = "Укажите телефон клиента";
        return false;
    }

    if (_reservation.Master is null)
    {
        validationError = "Выберите мастера";
        return false;
    }

    if (!TimeOnlyConverter.TryParse(StartTimeTextBox.Text, out _))
    {
        validationError = "Неверный формат времени. Используйте формат ЧЧ:ММ (например, 14:30).";
        return false;
    }

    return true;
}
```

Принципы валидации:

1. Проверяем каждое поле по очереди
2. При первой ошибке возвращаем `false` и текст ошибки
3. Выводим ошибку пользователю через `MessageBox`

```
if (!ValidateForm(out var validationError))
{
    MessageBox.Show(validationError, "Ошибка валидации", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
}
```

Настройка Git

Git — это система контроля версий, которая позволяет отслеживать изменения кода, откатываться к предыдущим версиям и работать в команде.

Установка Git

1. Скачайте Git с официального сайта: <https://gitforwindows.org/>
2. Установите Git с настройками по умолчанию

Настройка пользователя

После установки необходимо настроить имя и email, которые будут использоваться в коммитах:

```
git config --global user.name "Ваше Имя"
git config --global user.email "your.email@example.com"
```

Важно: Эти настройки выполняются один раз на компьютере и применяются ко всем репозиториям.

Инициализация репозитория

1. Откройте терминал (или Git Bash) в корневой папке вашего проекта (где находится `.sln` файл)
2. Инициализируйте Git-репозиторий:

```
git init
```

3. Скачайте готовый `.gitignore` файл для Visual Studio и C#:

- Перейдите на сайт: <https://www.toptal.com/developers/gitignore/api/visualstudio,csharp,dotnetcore>
- Скопируйте содержимое страницы
- Создайте файл `.gitignore` в корне проекта и вставьте скопированное содержимое

Что делает `.gitignore`:

- Исключает из Git служебные файлы Visual Studio (`bin/`, `obj/`, `.vs/`)
- Исключает артефакты сборки и пользовательские настройки
- Предотвращает загрузку ненужных файлов в репозиторий

Создание первого коммита

1. Добавьте `.gitignore` в репозиторий:

```
git add .gitignore
```

2. Добавьте все файлы проекта:

```
git add .
```

3. Создайте первый коммит:

```
git commit -m "Initial commit: создание проекта с базовыми классами и формами"
```

Проверка статуса

Чтобы посмотреть состояние репозитория, используйте:

```
git status
```

Просмотр истории коммитов

Чтобы посмотреть историю коммитов:

```
git log --oneline
```

Рекомендации по работе с Git

1. **Делайте коммиты часто** — после каждого логического изменения (добавили класс, реализовали функцию и т.д.)
2. **Пишите понятные сообщения коммитов:**
 - Хорошо: "Add Reservation class with validation"
 - Плохо: "Update", "Fix", "Changes"

3. **Используйте .gitignore** — не загружайте в репозиторий служебные файлы
 4. **Изучите основы Git** — прочитайте первые три главы официального руководства: <https://git-scm.com/book/ru/v2>
-

Заключение

В данной лекции мы рассмотрели:

1. **Основы WPF и XAML** — создание окон и элементов управления
2. **Архитектуру с разделением на слои** — Domain и UI проекты
3. **Классы предметной области** — Client, Master, Reservation, ReservationStatus
4. **Data Binding и DataContext** — автоматическая синхронизация данных
5. **Конвертеры значений** — преобразование типов для отображения в UI
6. **Валидация данных** — проверка корректности ввода
7. **Работу с Git** — инициализация репозитория и создание коммитов

Эти знания позволят вам создавать структурированные WPF-приложения с чистой архитектурой и эффективной работой с данными.

Дополнительные ресурсы

- Официальная документация WPF: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/>
- Руководство по Git: <https://git-scm.com/book/ru/v2>
- Документация C#: <https://learn.microsoft.com/ru-ru/dotnet/csharp/>